

COMP3910: Combinatorial Optimisation

Worksheet 7

Problem 1 (from A. Levitin “Introduction to the Design and Analysis of Algorithms”)

This exercise is on the Branch-and-Bound algorithm for the assignment problem. Note that the assignment problem is solvable in polynomial time by various efficient methods (the best one has the time complexity $O(n^3)$). The purpose of this exercise is to illustrate the Branch-and-Bound algorithm, although B&B cannot be recommended for a polynomially solvable problem.

In the assignment problem, n persons should be assigned to n jobs, one person per job. For each combination of a person and a job, the cost of assignment is given:

	job 1	job 2	job 3	job 4
person a	9	2	7	8
person b	6	4	3	7
person c	5	8	1	8
person d	1	6	9	4

The task is to find an optimal assignment of persons to jobs minimising the total cost.

Branching

Start with an empty solution $(*, *, *, *)$ (here $*$ in the k -th position indicates that no person has been assigned to job k , $1 \leq k \leq 4$). The first branching can be done by allocating one of the four persons to job 1: $(a, *, *, *)$; $(b, *, *, *)$; $(c, *, *, *)$; $(d, *, *, *)$. Next branching allocates one of the remaining three persons to job 2, etc.

Upper bounding

Consider solution (a, b, c, d) to define the first upper bound UB . From the above table, the assignment

person a to job 1,

person b to job 2,

person c to job 3,

person d to job 4,

has the cost $UB = 9 + 4 + 1 + 4 = 18$. Whenever a better solution is found with an objective value smaller than UB , replace UB by that value.

Lower bounding

Consider a partial solution $(i_1, \dots, i_k, *, \dots, *)$ obtained after k persons have been assigned to the first k jobs. To calculate LB , we find the sum of k cost-values for the assigned persons (taken from the above table) and the lowest costs for the remaining persons (the assignment cost cannot be less than the smallest element in the corresponding row).

For example, the lower bound for $(a, *, *, *)$ is $9 + (3 + 1 + 1) = 14$. Note that 14 does not correspond to a feasible assignment as persons b and c cannot be allocated to the same job (job 3), and persons a and d cannot be allocated to the same job (job 1).

The search tree for the described procedure is shown in Fig. 1. The nodes of the tree are numbered in the order they are constructed.

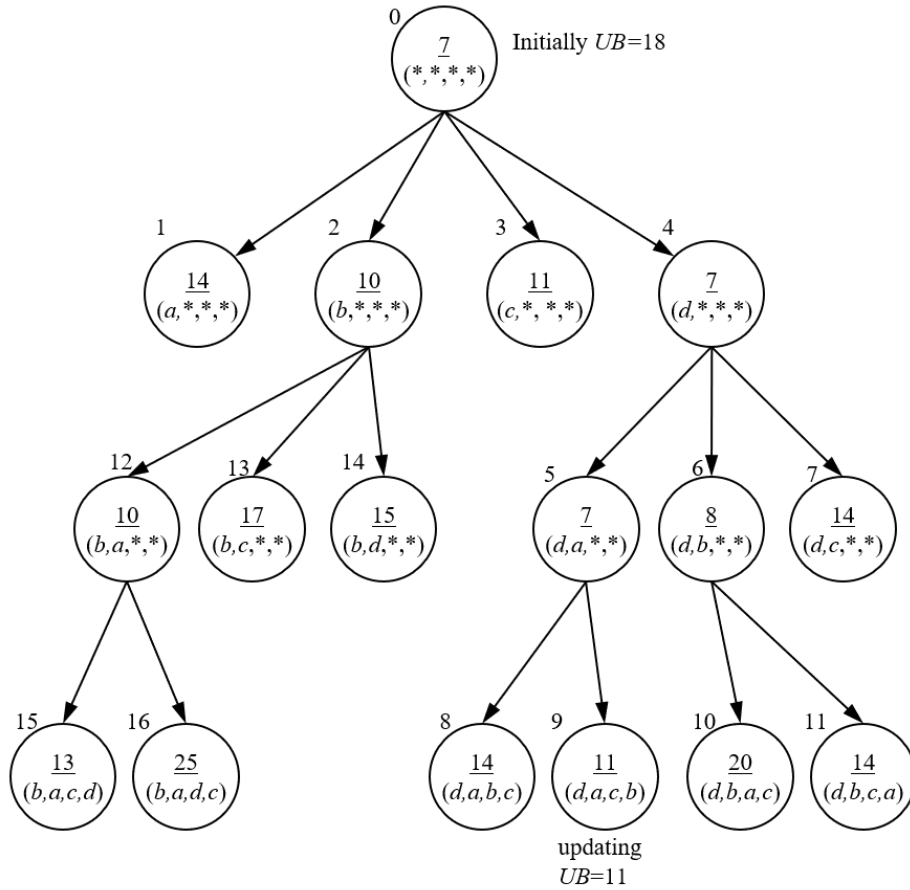


Figure 1: Search Tree for Problem 1

The optimal solution of cost 11 is (d, a, c, b) .

It defines the following assignment:

person d to job 1, person a to job 2, person c to job 3, person b to job 4.

(A) Verify the performance of the Branch-and-Bound algorithm.

- What strategy is adopted for selecting an active problem to be explored next? Note that for a minimisation problem, the next node for branching is selected by considering LBs. Before applying either strategy, depth-first or best-first search, all child-nodes of the current node should be produced (and their LBs should be calculated) prior to choosing the next node for branching. The depth first strategy considers the **most recent** nodes from the list of active nodes and chooses the one with the lowest LB (for min-problem), while the best-first search strategy considers the **whole list** of active nodes to select the one with the smallest lower bound.
- Check that the lower bounds are calculated as described on p. 1.

(B) How can we calculate tighter lower bounds? Suggest an improved formula for LB . Check whether this can reduce the search by recalculating the lower bounds of Fig. 1.

Problem 2 (from A. Levitin “Introduction to the Design and Analysis of Algorithms”)

An instance of the travelling salesman problem with $n = 5$ cities is given by the graph $G = (V, E)$ shown in Fig. 2. An optimal solution can be found by the branch-and-bound algorithm described in Lecture 9. There are a number of alternative versions of branch-and-bound for the TSP. One possible approach can be described as follows.

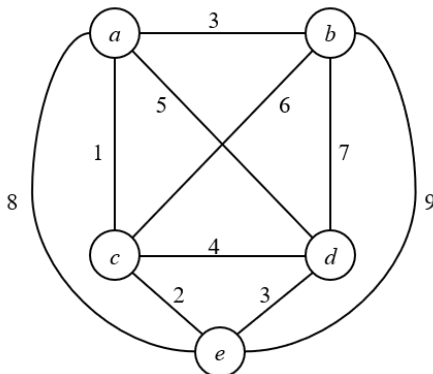


Figure 2: The input graph for Problem 2

Branching

The root corresponds to the partial solution $(a, *, *, *, a)$ with node s fixed as the starting and terminal node of the tour. We need to assign cities $\{b, c, d, e\}$ to the remaining 4 positions in the order they appear in the tour.

The first branching is done by fixing the second node of the tour. Then we branch by fixing the next node, etc.

Lower bounding

For each node i , $1 \leq i \leq n$, find the sum s_i of the distances to the two nearest nodes. Then sum up these n numbers, $s = \sum_{i=1}^n s_i$, and divide the sum by 2:

$$LB = s/2. \quad (1)$$

For example, the first lower bound for the instance shown in Fig. 2 is $LB = \underline{14}$ since

$$s_a = 1 + 3, \quad s_b = 3 + 6, \quad s_c = 1 + 2, \quad s_d = 3 + 4, \quad s_e = 2 + 3.$$

If at some iteration a particular edge connecting u and v has been fixed so that u is an immediate predecessor of v , the lower bound for all tours with the fixed edge (u, v) should be modified accordingly: s_u and s_v should take into account the fixed edge. For example, the lower bound of all tours starting with a, b, c is $LB_5 = \underline{16}$ defined as $(s_a + s_b + s_c + s_d + s_e)/2$ with the values

$$s_a = \boxed{3} + 1, \quad s_b = \boxed{3} + \boxed{6}, \quad s_c = \boxed{6} + 1, \quad s_d = 3 + 4, \quad s_e = 2 + 3.$$

Here the marked values corresponds to the fixed edges connecting a, b, c ; the remaining values correspond to the lengths of the shortest non-fixed edges.

Upper bounding

Consider (a, b, c, d, e, a) to define the first upper bound. Whenever a better solution is found with an objective value smaller than UB , we replace UB by this smaller value.

Since the graph is undirected, we can reduce the search by generating only tours in which b is visited before c eliminating the routes with the opposite order of these two nodes. In addition, we can stop branching once the first $n - 1$ nodes of a tour are fixed (the remaining node appears at the end of the tour, just before returning back to the origin).

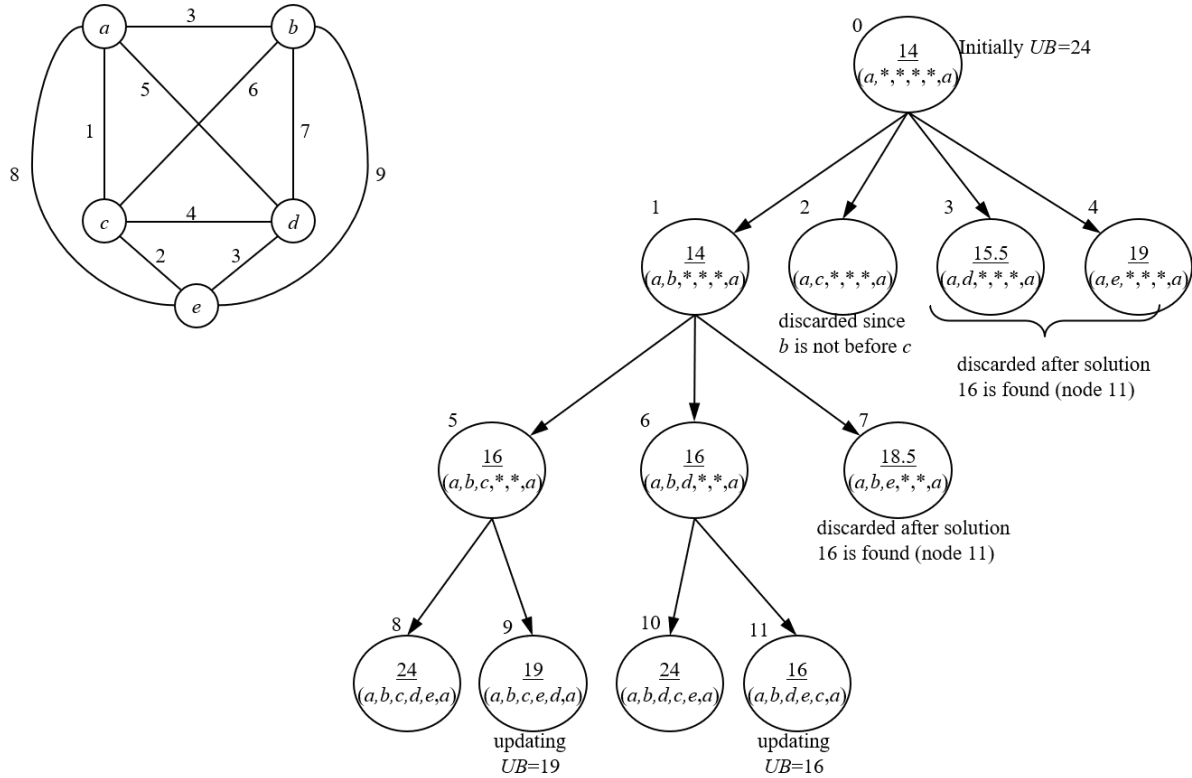


Figure 3: The search tree for Problem 2

The optimal solution is the tour (a, b, d, e, c, a) of length 16.

Questions

1. Justify the correctness of formula (1) for instances of the TSP with symmetric matrices.
2. How can we calculate tighter lower bounds? Suggest an improved formula for LB . Check whether this can reduce the search by recalculating the lower bounds of Fig. 3.